

IB Subject(s): Mathematics
Extended Essay

Group 5

Bezier Curves and curvature

How can the Bezier curve be used to plot the smoothest curve when an aircraft makes a medium level turn (constant altitude)?

Table of Contents

<i>Introduction:</i>	2
Why Bezier Curves?	2
Application in field of research:	4
<i>About Bezier Curves</i>	5
Introduction to Bezier curves:	5
Finding derivatives of Bezier Curves.....	8
Finding the curvature of a curve.....	12
Curvature of a Bezier Curve.....	18
<i>Application:</i>	20
Method 1.....	20
Method 2:	23
<i>Conclusion:</i>	27
Extension:.....	27
<i>References and Bibliography:</i>	28

Introduction:

Why Bezier Curves?

I first came across Bezier curves in the form of curve stitching in primary school, a concept of using straight lines stitched at regular intervals along two or more straight lines to form a curve. This concept intrigued me, and so I began reading up on how it worked and how supposed curved lines appeared in a work of art that contained only straight lines. This led to my discovery of Bezier curves.

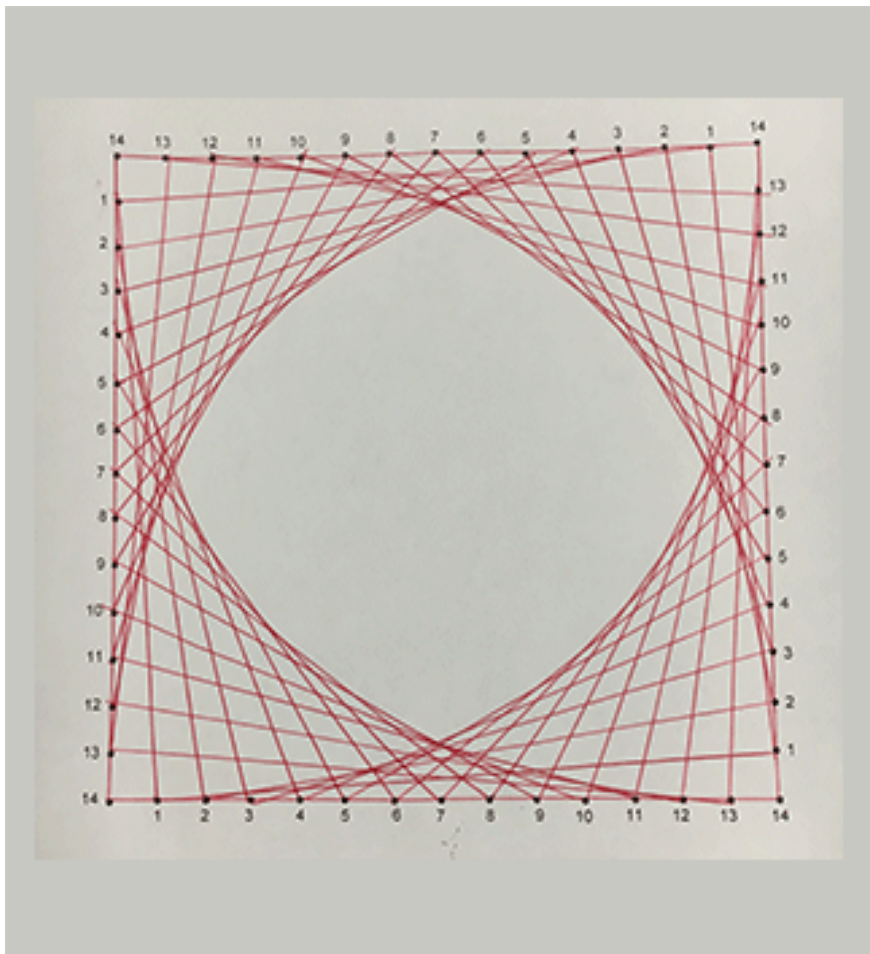


Figure 1 (Example of curve stitching, American Mathematical Society, 202))

Bezier curves are a concept that are similarly used in creating curves in most digital artworks so as to create smooth curves on a pixelated screen. These curves are often used in digital rendering and gaming, used for computer integrated graphics (CGI). Bezier curves are very widely used at present day: in large scale projects like the design of roller coasters, in the smaller scale graphic design in gaming, and even in the simple drawing tool in most software.

In doing my research, I hoped to be able to apply this concept to a part of my life that means a lot to me: flying. Some wide reading of articles taught me that Beziers curve had been used in the programming of autonomous vehicles, and this got me wondering if the same concept could be applied in flying. The autopilot system is today a common sight in most aircraft. However, most of the time, the autopilot merely maintains the altitude or bearing set by the pilot. I began to explore if it is possible for the autopilot to have a more sophisticated system allowing it to plan its own flight path instead. I realised that Bezier curves could provide a more optimised route for the planning of flight paths. A way that aircraft possibly plan the most optimised routes in flight. By optimised routes, I will be looking to see if Bezier curves provide less of a curvature than other methods, for if an aircraft had a turn of lower curvature, the bank angle or slant of the aircraft would be reduced, allowing for an increase in comfort for the passengers.

Application in field of research:

In this research, I will be researching the application of the Bezier curves in the form of autonomous vehicles in an aeroplane's autopilot system. In aviation, navigation is a big part of flying, and since there are no roads in the air, pilots and air traffic controllers have adopted a slightly different form of route planning. This is done through the use of waypoints, or markers on the ground that extend up infinitely. There are two forms of waypoints for flying a fly-by waypoint and a fly-over waypoint. Fly-over waypoints are waypoints that require a pilot to manoeuvre directly over when in flight, while fly-by waypoints are ones that merely require the pilot to perform a turn before reaching the waypoint, to head for the next waypoint.

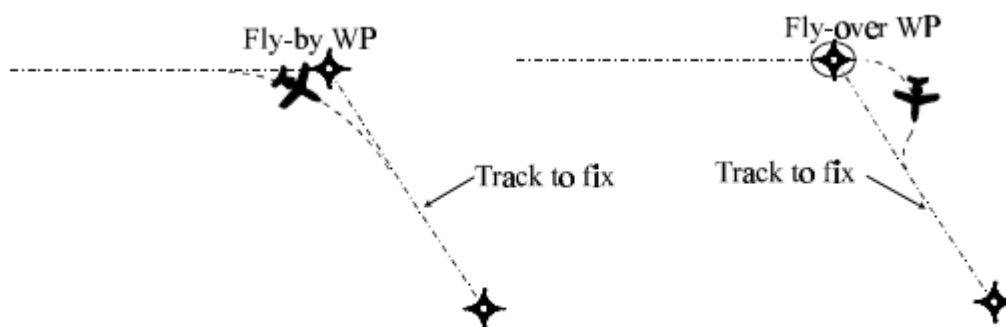


Figure 2 (Diagram showing airplane routes around waypoints, FAA, 2000)

In this essay, I will explore the application to Bezier curves of fly-by waypoints to chart suitable path for the plane. First, two fly-by waypoints two fly-over waypoints will be chosen. When an aircraft is flying in a certain direction, it must head directly for the waypoint of specific bearing, and when it completes the turn, it must fly over the fly-over waypoint.

In this research paper, I will be discussing and comparing two methods, in an effort to find the best path of flight.

About Bezier Curves

Introduction to Bezier curves:

Before explaining how they are used, it is first important to understand what Bezier curves are and how they can be derived. Bezier curves are a form of vectors that are plotted using parametric curves. These Bezier curves are just one out of the many classes of parametric functions. Parametric functions are functions of a number of coordinates (2 for the 2-dimensional plane, 3 for 3-D space, and so on), where each coordinate ($x, y, z \dots$) is expressed as another function of some parameter, like time: $x = f(t), y = g(t), z = h(t)$, and so on. (Xaktly, 2012)

With this information, parametric functions can then cause an x and y value to be independently determined through a separate variable, as demonstrated in the example below:

$$\begin{cases} x = \cos(t) \\ y = \sin(t) \end{cases}$$

where t is a mystery value that can change.

Each time the value of t is changed, we change two values, the x and the y value, which can then be plotted on a graph as coordinates.

Furthermore, unlike normal polynomials which use an x or y value to generate the other's outputs, Bezier curves make use of the binomial polynomial to produce both the x and y outputs. Bezier curves are a function of the polynomial t instead of the conventional x , where t is a value between fixed boundaries of 0 and 1. The coefficients of the functions take on the binomial form, which can be referenced from Pascal's triangle. 3 sample Bezier curve functions are shown below:

$$linear = (1 - t) + t$$

$$square = (1 - t)^2 + 2 * (t) * (1 - t) + t^2$$

$$cubic = (1 - t)^3 + 3 * (1 - t)^2 * t + 3 * (1 - t) * t^2 + t^3$$

If, however, we remove the coefficients of each of the parts in the function, and rename $(1 - t)$ as a and t as b , the functions can be rewritten as shown below:

$$linear = a + b$$

$$square = a * a + a * b + b * b$$

$$cubic = a * a * a + a * a * b + a * b * b + b * b * b$$

Comparing the above representation along with the fact that it is a binomial polynomial, the generic equation can be formed as shown below:

$$Bezier(n, t) = \sum_{i=0}^n \binom{n}{i} * (1 - t)^{n-i} * t^i$$

Where $\binom{n}{i}$ is the binomial term and $(1 - t)^{n-i} * t^i$ is the polynomial term, with n referring to the power the function is set at.

Bezier curves are interpolation functions, meaning that the curves take a set of points, and generate values within those points. This means that one will never be able to generate values outside the boundaries created by the specified points. The points can also be known as coordinates, and each point will affect the curve in a different way. These points can also be known as “weights” in the equation.

In order to change the curves, we need to first change the weights. This is because the weights are eventually multiplied into the parametric function to generate a graph. These weights, represented as w in the equation, can be added to the generic equation for Bezier curves, as shown below:

$$Bezier(n, t) = \sum_{i=0}^n \binom{n}{i} * (1 - t)^{n-i} * t^i * w_i$$

Where w_i refers to the weights. This may look complicated at first, but in reality w_0 is merely our first coordinate and w_n is our last coordinate, while everything in between is merely a controlling coordinate.

In introducing weights into our equations, the new equations for Bezier curves can be demonstrated as shown:

$$linear = (1 - t) * w_0 + t * w_1$$

$$square = (1 - t)^2 * w_0 + 2 * (t) * (1 - t) * w_1 + t^2 * w_2$$

$$cubic = (1 - t)^3 * w_0 + 3 * (1 - t)^2 * t * w_1 + 3 * (1 - t) * t^2 * w_2 + t^3 * w_3$$

Since we know that Bezier functions are parametric functions, we can take the cubic Bezier curve as an example, and rewrite it as a parametric function, making use of the equations above.

$$Bezier(3, t) = \begin{cases} x = (1 - t)^3(x_0) + 3(1 - t)^2(t)(x_1) + 3(1 - t)(t^2)(x_2) + t^3(x_3) \\ y = (1 - t)^3(y_0) + 3(1 - t)^2(t)(y_1) + 3(1 - t)(t^2)(y_2) + t^3(y_3) \end{cases}$$

With this equation, say we want a cubic curve that starts at (10,20), which is controlled by (20,30) and (30,35), and that ends at (35,15). The expression for this curve can be written in the equation:

$$Bezier(3, t) = \begin{cases} x = (1 - t)^3(10) + 3(1 - t)^2(t)(20) + 3(1 - t)(t^2)(30) + t^3(35) \\ y = (1 - t)^3(20) + 3(1 - t)^2(t)(30) + 3(1 - t)(t^2)(35) + t^3(15) \end{cases}$$

If plotted on a graph, it will look something similar to the diagram shown below:

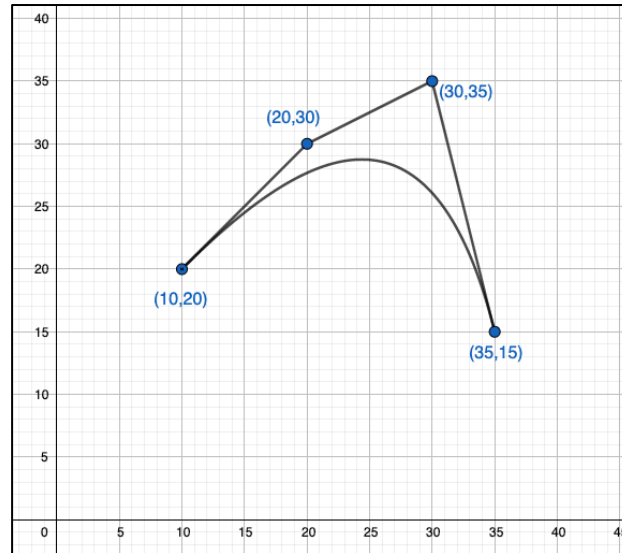


Figure 3 (Diagram showing cubic Bezier curve, Author's own, 2022)

The curved line seen joining the first and the last point is what is known as a Bezier curve.

Here is how I plotted the graph in figure 3. Firstly, implement a slider t where $(0 \leq t \leq 1)$.



Figure 4 (Diagram showing slider, Author's own, 2022)

Next, select 4 coordinates, and replace the weights with the coordinates in the equation shown below:

$$Bezier(3, t) = (1 - t)^3 * w_0 + 3 * (1 - t)^2 * t * w_1 + 3 * (1 - t) * t^2 * w_2 + t^3 * w_3$$

To plot the entire curve however, I made use of the Locus function on GeoGebra, so as to set all the points of the curve on a continuous line. This equation can be seen below:

$$Curve = Locus(Bezier(3, t), t)$$

This should then give you your desired Bezier curve.

Finding derivatives of Bezier Curves

Since the weights w are independent of the Bezier function, they can be removed temporarily from the equation. This, making use of the general equation, shows what is demonstrated below:

$$Bezier(n, t) = \sum_{i=0}^n \binom{n}{i} * (1-t)^{n-i} * t^i$$

$$Bezier'(n, t) = \frac{d}{dt} \sum_{i=0}^n \binom{n}{i} * (1-t)^{n-i} * t^i$$

However, this form of expressing the curve will be very tedious to work out, for as n increases, the amount of values in the summation increases. As such, we will take a sample for i where $k = i$, so as to find a general formula for the derivative of these terms. We will take this term of the summation expression to be $Bezier_k(n, t)$, with the formula where $k \in \mathbb{Z}$.

$$Bezier_k(n, t) = \binom{n}{k} t^k (1-t)^{n-k}$$

$$Bezier'_k(n, t) = \frac{d}{dt} \binom{n}{k} t^k (1-t)^{n-k}$$

By applying chain rule and product rule, we can get the next step,

$$\dots = \binom{n}{k} [k * t^{k-1} * (1-t)^{n-k} + t^k * (1-k) * (1-t)^{n-k-1} * (n-k) * -1]$$

Expanding $\binom{n}{k}$, we get:

$$\dots = \frac{n!}{k! (n-k)!} * [k * t^{k-1} * (1-t)^{n-k} + t^k * (1-k) * (1-t)^{n-k-1} * (n-k) * -1]$$

$$\dots = \frac{kn!}{k! (n-k)!} * t^{k-1} * (1-t)^{n-k} - \frac{(n-k)n!}{k! (n-k)!} * t^k * (1-t)^{n-k-1}$$

$$\dots = \frac{n!}{(k-1)! (n-k)!} * t^{k-1} * (1-t)^{n-k} - \frac{(n-k)n!}{k! (n-k)!} * t^k * (1-t)^{n-k-1}$$

Following this, we want to change the equations back to a binomial form, so we can end up with something similar to “ $x!/y! (x-y)!$ ”.

$$\dots = n \left(\frac{(n-1)!}{(k-1)! (n-k)!} * t^{k-1} * (1-t)^{n-k} - \frac{(n-k)(n-1)!}{k! (n-k)!} * t^k * (1-t)^{n-k-1} \right)$$

$$\dots = n \left(\frac{(n-1)!}{(k-1)!((n-1)-(k-1))!} * t^{k-1} * (1-t)^{((n-1)-(k-1))} - \frac{(n-1)!}{k!((n-1)-k)!} * t^k * (1-t)^{(n-1)-k} \right)$$

If we substitute $x = n - 1$ and $y = k - 1$ into the equations, we can see a result that is very similar to the original representation of 2 Bezier curves.

$$\dots = n \left(\frac{x!}{y!x-y!} * t^y * (1-t)^{x-y} - \frac{x!}{k!(x-k)!} * t^k * (1-t)^{x-k} \right)$$

$$\dots = n \left(Bezier_y(x, t) - Bezier_k(x, t) \right)$$

Since $x = n - 1$ and $y = k - 1$,

$$\dots = n \left(Bezier_{k-1}(n-1, t) - Bezier_k(n-1, t) \right)$$

Reapplying this back to our weighted Bezier curves, the equation can then be expanded.

$$Bezier(n, t) = Bezier_0(n, t) * w_0 + Bezier_1(n, t) * w_1 + Bezier_2(n, t) * w_2 + Bezier_3(n, t) * w_3$$

$$+ \dots$$

$$Bezier'(n, t) = n \left(Bezier_{-1}(n-1, t) - Bezier_0(n-1, t) \right) * w_0$$

$$+ n \left(Bezier_0(n-1, t) - Bezier_1(n-1, t) \right) * w_1$$

$$+ n \left(Bezier_1(n-1, t) - Bezier_2(n-1, t) \right) * w_2$$

$$+ n \left(Bezier_2(n-1, t) - Bezier_3(n-1, t) \right) * w_3 + \dots$$

In rewriting this as shown below, a pattern can be observed:

$$n * Bezier_{-1}(n-1, t) * w_0 +$$

$$n * Bezier_0(n-1, t) * w_1 - n * Bezier_0(n-1, t) * w_0 +$$

$$n * Bezier_1(n-1, t) * w_2 - n * Bezier_1(n-1, t) * w_1 +$$

$$n * Bezier_2(n-1, t) * w_3 - n * Bezier_3(n-1, t) * w_2 +$$

$$\dots - n * Bezier_3(n-1, t) * w_3 +$$

$$\dots$$

From this equation of the derivative, two of the terms can be removed, the first and the last. This is because there is no -1^{st} term in a summation, meaning that the first term will contribute to nothing, and thus can be ignored for the purpose of finding the derivative function. The last term can be removed since it is one involving $Bezier_n(n-1, t)$, meaning that it would have a non-existent binomial coefficient of $[i \text{ choose } i+1]$. As such, this term would contribute to “nothing”, and can hence be excluded for further calculations. With this in mind, we are left with the following:

$$\begin{aligned} & n * Bezier_0(n-1, t) * w_1 - n * Bezier_0(n-1, t) * w_0 + \\ & n * Bezier_1(n-1, t) * w_2 - n * Bezier_1(n-1, t) * w_1 + \\ & n * Bezier_2(n-1, t) * w_3 - n * Bezier_3(n-1, t) * w_2 + \\ & \dots \end{aligned}$$

This is just a mere summation of the lower order curves, and can hence be rewritten as shown below:

$$\begin{aligned} Bezier'(n, t) = & n * Bezier_0(n-1, t) * (w_1 - w_0) + n * Bezier_1(n-1, t) * (w_2 - w_1) + \\ & n * Bezier_2(n-1, t) * (w_3 - w_2) + \dots \end{aligned}$$

In rewriting this as a normal summation, we can produce the equation shown below, and we are done.

$$Bezier'(n, t) = \sum_{n=0}^{n-1} n * Bezier_k(n-1, t) * (w_{k+1} - w_k) = \sum_{n=0}^{n-1} Bezier_k(n, t) * n * (w_{k+1} - w_k)$$

This brings us back to our original formula of our Bezier curve, and we can notice the differences between our equation and the equation of the derivative.

$$\begin{aligned} Bezier(n, t) &= \sum_{i=0}^n \binom{n}{i} * (1-t)^{n-i} * t^i * w_i \\ Bezier'(n, t) &= \sum_{i=0}^{n-1} \binom{n-1}{i} * (1-t)^{n-1-i} * t^i * n * (w_{i+1} - w_i) \end{aligned}$$

In comparing the two Bezier curves, there are virtually no differences in the functions themselves, but only a change to the weight of the curves. The order of the curves is also reduced by 1, along with the number of points. For example, if the curve had four points, W, X, Y and Z , the first derivative would have 3 points, while the second would have 2, and the third would have 1. In

function of the derivative of the Bezier curve, the new derivative weight is in the term $[t^i * n * (w_{i+1} - w_i)]$. As such, the derivative weights are demonstrated below:

$$\text{Bezier}(n, t) \quad w = (W, X, Y, Z)$$

$$\text{Bezier}'(n, t) \quad n = 3 \quad w = (W', X', Y') = \{3 * (X - W), 3 * (Y - X), 3 * (Z - Y)\}$$

$$\text{Bezier}'(n, t) \quad n = 2 \quad w = (W'', X'') = \{2 * (X' - W'), 2 * (Y' - X')\}$$

$$\text{Bezier}'(n, t) \quad n = 1 \quad w = (W''') = \{1 * (X'' - W'')\}$$

This process can be carried on for as long as there is more than one weight. Once there is only one weight left, the next step will have $k = 0$, and this will result in the summation of the function equalling to 0. This is because nothing is being added at all.

Finding the curvature of a curve

To visualise curvature, one must first imagine that a curve of changing curvatures is plotted. As an example, the parametric function shown below demonstrates a curve of changing curvatures. We can express this parametric function as vector function s that takes in a single function t , with the output values x and y , each as a function of t . Next, we parameterise the function by $t - \sin(t)$ and $1 - \cos(t)$, and this written below as the function:

$$\vec{s}(t) = \begin{bmatrix} x(t) \\ y(t) \end{bmatrix} = \begin{bmatrix} t - \sin(t) \\ 1 - \cos(t) \end{bmatrix}$$

If plotted in on a cartesian graph, the diagram below can be observed.

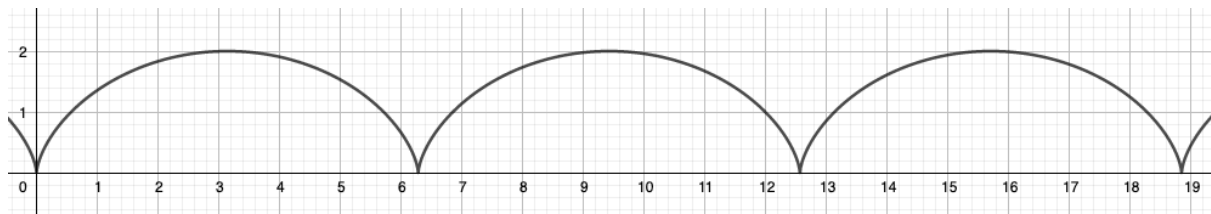


Figure 5 (Graph demonstrating curve of changing curvatures, Author's own, 2022)

To understand what is being written as curvature here, imagine the graph shown to be a road. If one were to drive along this road, he would be constantly turning at a different rate. However, if at any point along the road his steering wheel were to lock, the circle one drew would be defined as the radius of curvature, or R . From this, the curvature, which can be denoted as the Greek alphabet kappa (κ), is represented in the equation $R = \frac{1}{\kappa}$.

Firstly, to find curvature, one needs to find the tangent vectors. Imagine a unit tangent vector at every given point along the curve. One might consider how fast these unit tangent vectors change direction. In the little schematic drawn in Fig. 6, 3 sample tangent vectors T_1 , T_2 and T_3 will be drawn on the graph. While doing this, we will assume that all tangent vectors have a unit length $\|T_i\| = 1$.

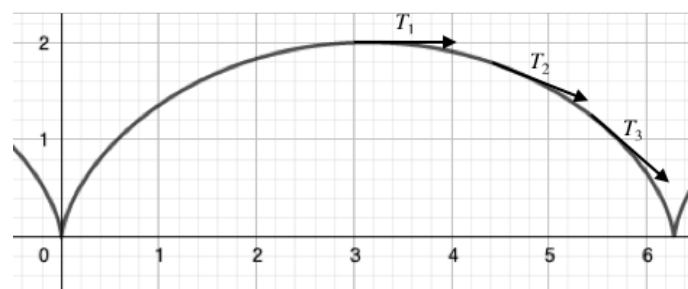


Figure 6 (Diagram showing tangent vectors on curve, Author's own, 2022)

One might imagine in a completely different space, instead of rooting each vector to the curve, one could write the vector in its own right, off in some other spot. This will allow it to be easier to visualise how these vectors turn as the graph moves from left to right, as demonstrated below:

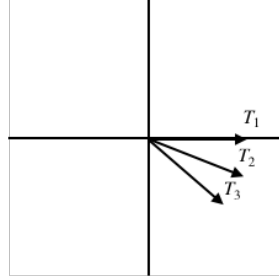


Figure 7 (Diagram showing tangent vectors rooted at the same position, Author's own, 2022)

When we observe the changes in unit tangent vector from T_1 to T_2 and subsequently T_3 , this change large or small, can be defined as the rate in change of the unit tangent vector, which can be expressed as dT .

So, the idea behind curvature is that we are going to take the rate of change of the unit tangent vector dT divided by the rate of change of arc length, ds , where s refers to the arc length or the distance of the small step taken between the vectors. So, what we want to know in curvature, is how much this unit tangent vector changes, as one moves along the arc length. With this knowledge, the formula for curvature can be plotted, as shown below:

$$\kappa = \frac{dT}{ds}$$

Going back to the changes in unit tangent vector, these changes are going to be some kinds of vectors as well. Because the curvature is actually just a number to be found, what we actually want is the size of the vector, which can be denoted as follows:

$$\kappa = \left\| \frac{dT}{ds} \right\|$$

To further understand the equation, let us take another example, with the parameterisation as shown below:

$$\vec{r}_s(t) = \begin{bmatrix} x(t) \\ y(t) \end{bmatrix} = \begin{bmatrix} \cos(t)R \\ \sin(t)R \end{bmatrix} \quad R \in \mathbb{R}$$

This will result in a circle drawn on the cartesian plane with radius $R = 1$ as shown below:

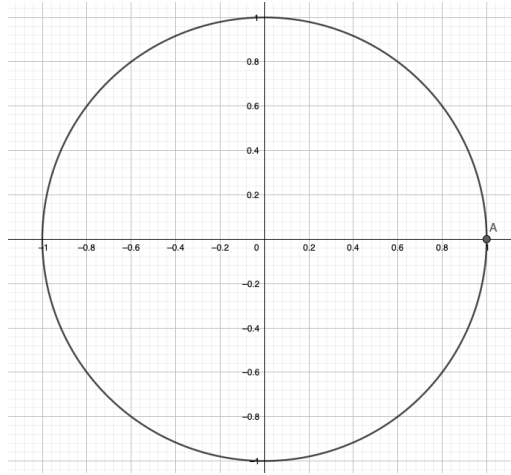


Figure 8 (Graph showing the circle with radius R , Author's own, 2022)

Since the tangent vector to this curve is basically the gradient of the curve, this can be expressed mathematically in the equation

$$\vec{s}'(t) = \begin{bmatrix} -\sin(t)R \\ \cos(t)R \end{bmatrix} \quad R \in \mathbb{R}$$

However, this only gives us the tangent vector, which may not be a unit tangent vector. Since we are looking for a unit tangent vector, we need to normalise it and get a unit tangent vector function, which will be called $T(t)$.

$$T(t) = \frac{\vec{s}'(t)}{\|\vec{s}'(t)\|}$$

In this case, the magnitude of this derivative as a function of t can be calculated as follows

$$\begin{aligned} & \|\vec{s}'(t)\| \\ &= \left\| \begin{bmatrix} -\sin(t)R \\ \cos(t)R \end{bmatrix} \right\| \\ &= \sqrt{\sin^2(t)R^2 + \cos^2(t)R^2} \\ &= R\sqrt{\sin^2(t) + \cos^2(t)} \\ &= R\sqrt{1} \\ &= R \end{aligned}$$

Substituting it back into the equation for calculation of unit tangent vectors, the unit tangent vector can then be more easily found.

$$T(t) = \frac{\vec{s}'(t)}{R}$$

$$T(t) = \frac{\begin{bmatrix} -\sin(t)R \\ \cos(t)R \end{bmatrix}}{R}$$

$$T(t) = \begin{bmatrix} -\sin(t) \\ \cos(t) \end{bmatrix}$$

Since the derivative of $T(t)$ is written as $T'(t)$, it can also be expressed in the form $\frac{dT}{dt}$.

Hence, using the previous working, it can be found that

$$\frac{dT}{dt} = \begin{bmatrix} -\cos(t) \\ -\sin(t) \end{bmatrix}$$

This implies that the magnitude of this vector can be expressed as:

$$\left\| \frac{dT}{dt} \right\| = \sqrt{\sin^2(t) + \cos^2(t)}$$

$$\left\| \frac{dT}{dt} \right\| = 1$$

Before going any further, it is also important to understand what this looks like abstractly, without the parameterisation. This will be represented as shown below, along with its derivative:

$$\vec{s}(t) = \begin{bmatrix} x(t) \\ y(t) \end{bmatrix}$$

$$\vec{s}'(t) = \begin{bmatrix} x'(t) \\ y'(t) \end{bmatrix}$$

Since we also know the formula for the unit tangent vector, we can hence input these into the equation, as shown.

$$T(t) = \frac{\vec{s}'(t)}{\left\| \vec{s}'(t) \right\|}$$

However, here is where things change from the circle, because the magnitude of this derivative is not as easily presented. This magnitude is shown in the equation:

$$\frac{\left\| \vec{r}'(t) \right\|}{\sqrt{x'(t)^2 + y'(t)^2}}$$

When substituted back into the equation, the formula for unit tangent vector can be shown as

$$T(t) = \frac{\vec{r}'(t)}{\left\| \vec{r}'(t) \right\|} = \frac{\begin{bmatrix} x(t) \\ y(t) \end{bmatrix}}{\sqrt{x'(t)^2 + y'(t)^2}}$$

$$T(t) = \begin{bmatrix} x(t)/\sqrt{x'(t)^2 + y'(t)^2} \\ y(t)/\sqrt{x'(t)^2 + y'(t)^2} \end{bmatrix}$$

Going back to the equation of curvature, although we do not know the rate at which T changes with respect to s , we do directly know its change with respect to a tiny change in that perimeter.

$$\kappa = \left\| \frac{dT}{ds} \right\|$$

$$k = \left\| \frac{dT/dt}{d\vec{r}/dt} \right\|$$

With this knowledge, we can once again return to the equation involving the circle, as we have found that

$$\left\| \frac{dT}{dt} \right\| = 1$$

And

$$\left\| \frac{d\vec{r}}{dt} \right\| = R$$

As such, by inputting these values into the equation for curvature, we can hence tell that the curvature kappa (κ) can be expressed in the equation:

$$\kappa = \frac{1}{R}$$

This proves the equation introduced in the beginning that curvature κ is written as 1 divided by the radius and vice versa.

However, this is not the formula for curvature, but only the formula to find the radius of curvature. The actual formula for curvature has to make use of the abstract equation to find how to express curvature using parametric x and y functions. Since we know that $T(t)$ is, we can find out what $\frac{dT}{dt}$ is, by finding the derivative of $T(t)$. The magnitude of this will then be divided by the magnitude of the derivative of the original function $d\vec{r}_s$, to produce

$$\frac{\|dT/dt\|}{\|ds/dt\|} = \left\| \frac{dT}{ds} \right\| = \kappa$$

The calculations are then made using the parametric functions given and what is derived for $T(t)$. If simplified correctly, this should eventually lead to the equation that is shown below:

$$\kappa = \frac{x'(t)y''(t) - y'(t)x''(t)}{(x'(t)^2 + y'(t)^2)^{3/2}}$$

Curvature of a Bezier Curve

To find the curvature of a curve, we must first go back to the basic equation of a Bezier curve. For this example, we will be using a cubic Bezier curve, with the coordinates w_0, w_1, w_2 and w_3 , where $w_0 = (x_0, y_0)$, $w_1 = (x_1, y_1)$, $w_2 = (x_2, y_2)$ and $w_3 = (x_3, y_3)$. We know the Bezier function is expressed as

$$Bezier(n, t) = \sum_{i=0}^n \binom{n}{i} * (1-t)^{n-i} * t^i * w_i$$

and the graph is cubic, $n = 3$. Since we are starting by looking for the x components, we will use x_i instead of w_i . We will label this x component as $Bezier_x(3, t)$. With the knowledge of how to find the derivatives of Bezier functions and how to split a function of a Bezier curve, the following can be derived:

$$Bezier_x(3, t) = \sum_{i=0}^3 \binom{3}{i} * (1-t)^{3-i} * t^i * x_i$$

$$Bezier'_x(3, t) = \sum_{i=0}^2 \binom{2}{i} * (1-t)^{2-i} * t^i * 3 * (x_{i+1} - x_i)$$

$$Bezier''_x(3, t) = \sum_{i=0}^1 \binom{1}{i} * (1-t)^{1-i} * t^i * 2 * 3 * ((x_{i+2} - x_{i+1}) - (x_{i+1} - x_i))$$

$$Bezier''_x(3, t) = \sum_{i=0}^1 \binom{1}{i} * (1-t)^{1-i} * t^i * 2 * 3 * (x_{i+2} - x_i)$$

If these workings are expanded, the following is the result:

$$Bezier_x(3, t) = (1-t)^3(x_0) + 3(1-t)^2(t)(x_1) + 3(1-t)(t^2)(x_2) + t^3(x_3)$$

$$Bezier'_x(3, t) = 3 * (1-t)^2(x_1 - x_0) + 3 * 2 * (1-t)(t)(x_2 - x_1) + 3 * (t^2)(x_3 - x_2)$$

$$Bezier'_x(3, t) = 3(1-t)^2(x_1 - x_0) + 6(1-t)(t)(x_2 - x_1) + 3(t^2)(x_3 - x_2)$$

$$Bezier''_x(3, t) = 2 * (1-t)((x_2 - x_1) - (x_1 - x_0)) + 2 * (t)((x_3 - x_2) - (x_2 - x_1))$$

$$Bezier''_x(3, t) = 2(1-t)(x_2 - x_0) + 2(t)(x_3 - x_1)$$

Since the x and y functions are identical, the derivation of the equations for the y component of the graph is the same as the x component of the graph. As such, the equations and derivatives of the y component of the Bezier curve are shown below, where $Bezier_y(3, t)$ refers to the y component of the cubic function.

$$Bezier_y(3, t) = (1 - t)^3(y_0) + 3(1 - t)^2(t)(y_1) + 3(1 - t)(t^2)(y_2) + t^3(y_3)$$

$$Bezier_y'(3, t) = 3(1 - t)^2(y_1 - y_0) + 6(1 - t)(t)(y_2 - y_1) + 3(t^2)(y_3 - y_2)$$

$$Bezier_y''(3, t) = 2(1 - t)(y_2 - y_0) + 2(t)(y_3 - y_1)$$

In understanding the curvature of the curve, we know that the equation used to calculate the curvature of a curve is

$$\kappa = \frac{x'(t)y''(t) - y'(t)x''(t)}{(x'(t)^2 + y'(t)^2)^{3/2}}$$

In a Bezier curve however, since the x and y functions revolve around the term t , as shown when breaking down the equation, the formula used to find the curvature of a curve can also be expressed as shown below:

$$k = \frac{Bezier_x^{(3,t)} * Bezier_y''(3, t) - Bezier_x''(3, t) * Bezier_y'(3, t)}{(Bezier_x'(3, t)^2 + Bezier_y'(3, t)^2)^{3/2}}$$

With this in mind, we can hence find the curvature of any cubic Bezier curve, as long as we have the four coordinates of the curve. In this example that we have done, using the four coordinates (x_0, y_0) , (x_1, y_1) , (x_2, y_2) and (x_3, y_3) , the final curvature equation should look as shown below:

$$\begin{aligned} k = & \langle [3(1 - t)^2(x_1 - x_0) + 6(1 - t)(t)(x_2 - x_1) + 3(t^2)(x_3 - x_2)] \\ & * [2 * (1 - t)(y_2 - y_0) + 2 * (t)(y_3 - y_1)] \\ & - [2 * (1 - t)(x_2 - x_0) + 2 * (t)(x_3 - x_1)] \\ & * [3(1 - t)^2(y_1 - y_0) + 6(1 - t)(t)(y_2 - y_1) + 3(t^2)(y_3 - y_2)] \rangle \\ & \div \langle [3(1 - t)^2(x_1 - x_0) + 6(1 - t)(t)(x_2 - x_1) + 3(t^2)(x_3 - x_2)]^2 \\ & + [2(1 - t)(y_2 - y_0) + 2(t)(y_3 - y_1)]^2 \rangle^{3/2} \end{aligned}$$

Although this formula may seem very complex, when the actual literature coordinates of x and y are entered into the data set, the equation will be simplified, and the result can be plotted in a k/t graph, of t values $0 \leq t \leq 1$.

Application:

Method 1

With knowledge now of how to plot Bezier curves and how to find the curvature of the curves, I tested this method on some waypoints that I got from my time in flying. In the satellite image shown below, the four waypoints can be seen. These four waypoints were the points used by pilots when exiting the airport's airspace to join the training airspace. Throughout this route, the pilots had to ensure that they maintained level flight, which is important in this investigation as well, as this essay only explores a two-dimensional plane.

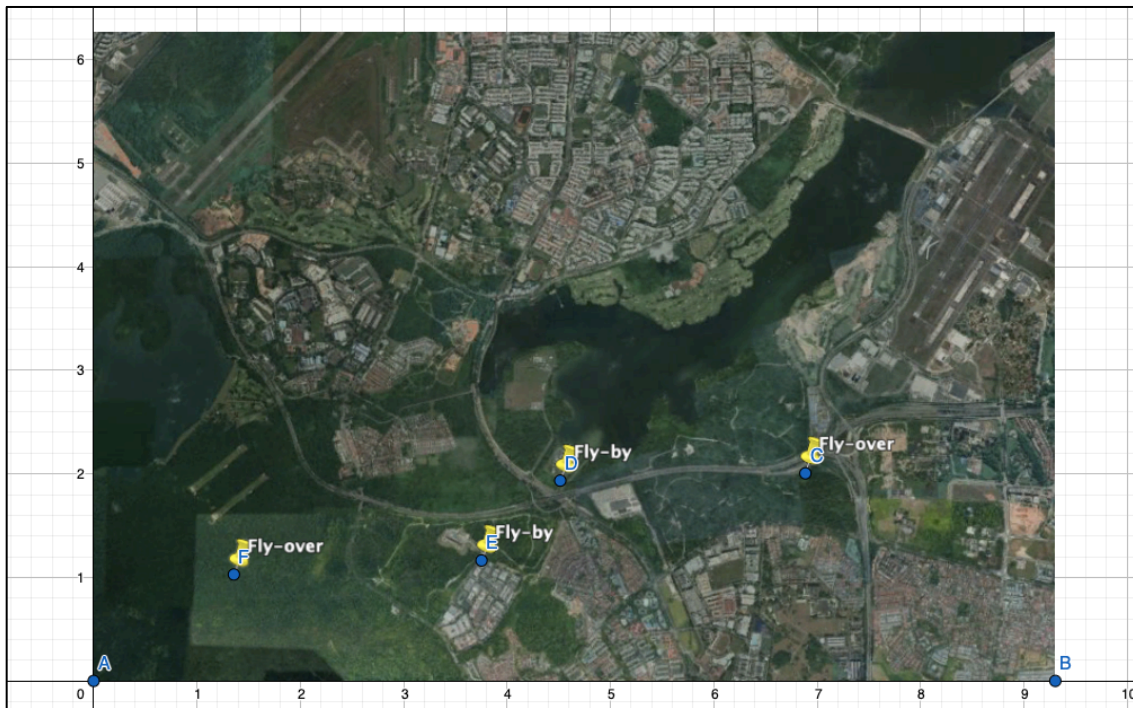


Figure 9 (Map showing 4 waypoints, Author's own, 2022)

Since the flight path was from C to F, the coordinates of C will represent w_0 , while D will be w_1 , E will be w_2 and F will be w_3 . The exact coordinates of C, D, E and F are (6.89,2.00), (4.51,1.93), (3.75,1.16) and (1.35,1.02) respectively. With this information, the Bezier curve can be represented in the equation:

$$B(t) = \begin{cases} B_x(t) = (1-t)^3(6.89) + 3(1-t)^2(t)(4.51) + 3(1-t)(t^2)(3.75) + t^3(1.35) \\ B_y(t) = (1-t)^3(2.00) + 3(1-t)^2(t)(1.39) + 3(1-t)(t^2)(1.16) + t^3(1.02) \end{cases}$$

With these coordinates, we are able to plot the Bezier curve on the graph, and find the derivatives, as demonstrated below:

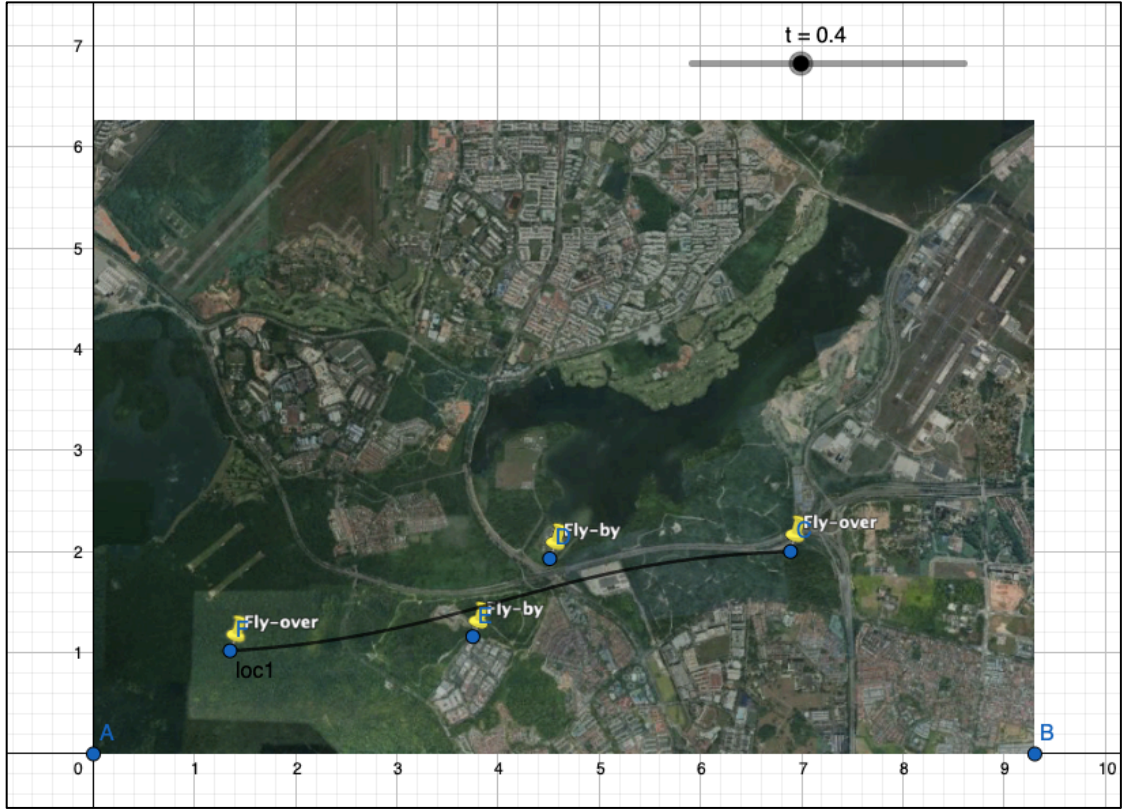


Figure 10 (Map showing Bezier curve along 4 waypoints, Author's own, 2022)

With the equation of this curve shown as above, we can verify how optimal the graph is. Since we have the values for the coordinates of the four points, we can substitute these coordinates into the equation for finding the curvature of Bezier curves. The original equation is shown to be:

$$\begin{aligned}
 k = & \langle [3(1-t)^2(x_1 - x_0) + 6(1-t)(t)(x_2 - x_1) + 3(t^2)(x_3 - x_2)] \\
 & * [2 * (1-t)(y_2 - y_0) + 2 * (t)(y_3 - y_1)] \\
 & - [2 * (1-t)(x_2 - x_0) + 2 * (t)(x_3 - x_1)] \\
 & * [3(1-t)^2(y_1 - y_0) + 6(1-t)(t)(y_2 - y_1) + 3(t^2)(y_3 - y_2)] \rangle \\
 & \div \langle [3(1-t)^2(x_1 - x_0) + 6(1-t)(t)(x_2 - x_1) + 3(t^2)(x_3 - x_2)]^2 \\
 & + [2(1-t)(y_2 - y_0) + 2(t)(y_3 - y_1)]^2 \rangle^{\frac{3}{2}}
 \end{aligned}$$

And after substituting the values of the coordinates, the equation be as follows:

$$\begin{aligned}
k &= \langle [3(1-t)^2(4.51 - 6.89) + 6(1-t)(t)(3.75 - 4.51) + 3(t^2)(1.35 - 3.75)] \\
&\quad * [2 * (1-t)(1.16 - 2.00) + 2 * (t)(1.02 - 1.93)] \\
&\quad - [2 * (1-t)(3.75 - 6.89) + 2 * (t)(1.35 - 4.51)] \\
&\quad * [3(1-t)^2(1.93 - 2.00) + 6(1-t)(t)(1.16 - 1.93) + 3(t^2)(1.02 - 1.16)] \rangle \\
&\quad \div \langle [3(1-t)^2(4.51 - 6.89) + 6(1-t)(t)(3.75 - 4.51) + 3(t^2)(1.35 - 3.75)]^2 \\
&\quad + [2(1-t)(1.16 - 2.00) + 2(t)(1.02 - 1.93)]^2 \rangle^{\frac{3}{2}} \\
k &= \langle [-7.14(1-t)^2 - 4.56(1-t)(t) - 7.2(t^2)] * [-1.68(1-t) - 1.82(t)] \\
&\quad - [-6.28(1-t) - 6.32(t)] * [-0.21(1-t)^2 - 4.62(1-t)(t) - 0.28(t^2)] \rangle \\
&\quad \div \langle [-7.14(1-t)^2 - 4.56(1-t)(t) - 7.2(t^2)]^2 + [-1.68(1-t) - 1.82(t)]^2 \rangle^{\frac{3}{2}}
\end{aligned}$$

This can be represented in a graph, where $k = y$ and $t = x$, to form the graph shown below:

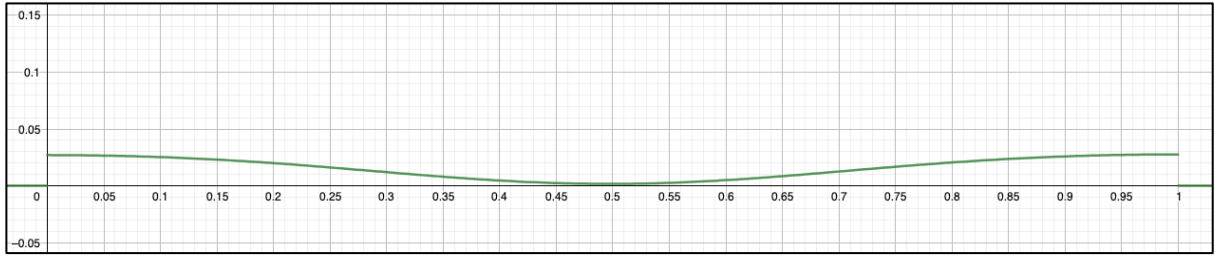


Figure 11 (Graph showing curvature against t for Bezier curve, Author's own, 2022)

From this graph, it can be seen that the optimised path has a very low curvature, with a maximum curvature of 0.0276727239883 when the graph is at its maximum. This indicates that the radius of curvature is also fairly large, at 13 units, indicating a very gradual turn in the curve. The general curvature is also very minimal, decreasing as it approaches 0.5 and increasing again as it moves away from 0.5.

Method 2:

For this method, I made use of the same map from GeoGebra, but this time incorporated 5 new coordinates, as shown in the figure below.

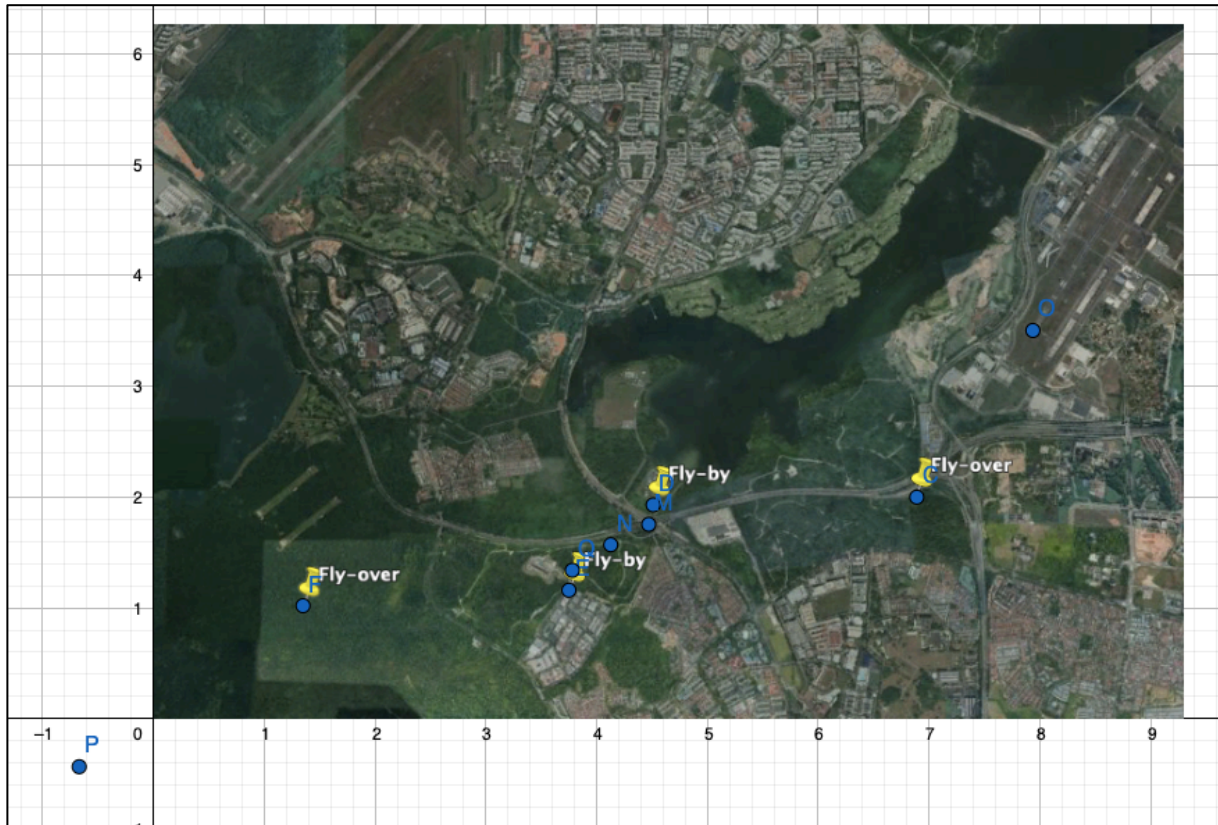


Figure 12 (Map showing 4 waypoints and 5 new coordinates, Author's own, 2022)

O and P represents the inbound and outbound point of the aircraft. M and Q represent the positions at which the aircraft should pass by in manual flight, since the pilot will not fly over the fly-by waypoints. N represents the mid-point between D and E, which are the two fly-by waypoints. As such, I decided to see if I could find the most optimal path from C through N and N through P using cubic “Fit” functions on GeoGebra. The result is demonstrated as shown in figure 13:



Figure 13 (Map demonstrating 2 cubic curves as best fit lines, Author's own, 2022)

However, since the coordinates P and O are not needed in this investigation, but serving only as references, they can be removed, and the graphs can be bound by the two fly-over waypoints.

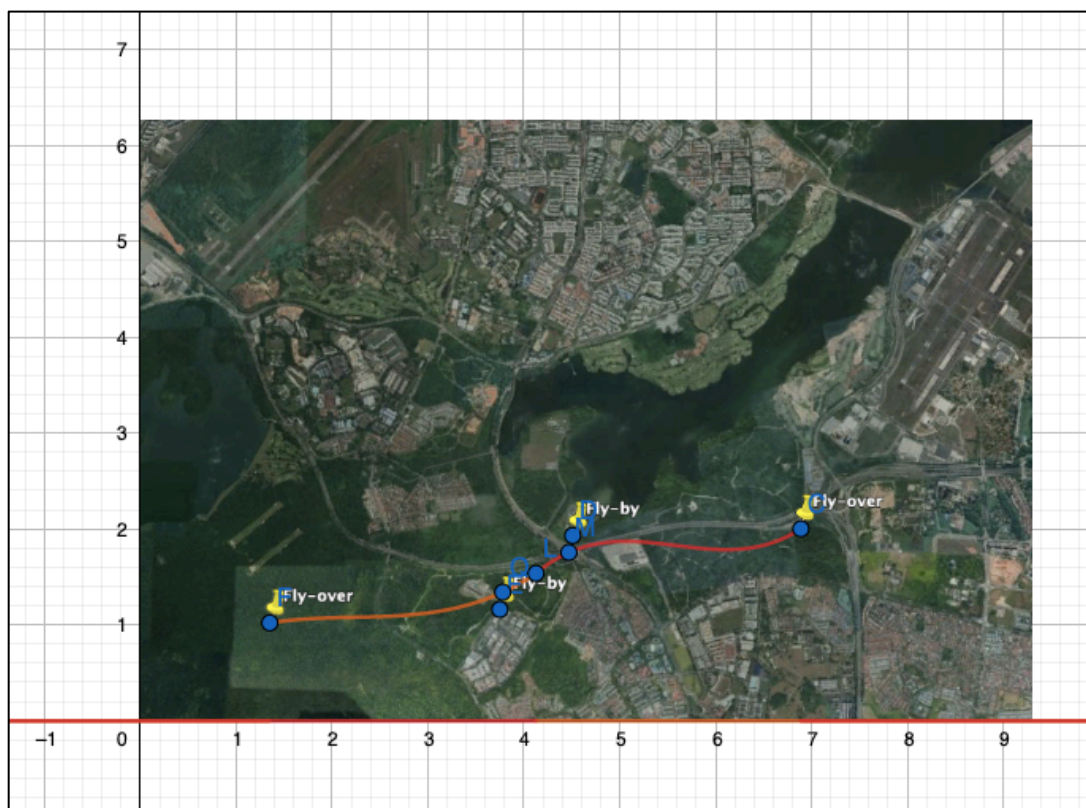


Figure 14 (Map demonstrating 2 cubic curves bound by fly-over waypoints, Author's own, 2022)

The equations of the graphs demonstrated here are shown below (the first is of the orange graph and the second is of the red graph):

$$f(x) = (0.0600222176602x^3 - 0.3997242887216x^2 + 0.9089627063146x + 0.3737206988949) (1.35 < x < 4.1275)$$

$$g(x) = (0.1516998427429x^3 - 2.5427875067454x^2 + 14.0830149746564x - 23.9389149976357) (4.1275 < x < 6.89)$$

With these two functions, we can find the first and second derivative of these functions, through simple differentiation:

$$f'(x) = (0.1800666529806x^2 - 0.7994485774432x + 0.9089627063146) (1.35 < x < 4.1275)$$

$$f''(x) = (0.3601333059612x - 0.7994485774432) (1.35 < x < 4.1275)$$

$$g'(x) = (0.4550995282287x^2 - 5.0855750134908x + 14.0830149746564) (4.1275 < x < 6.89)$$

$$g''(x) = (0.9101990564574x - 5.0855750134908) (4.1275 < x < 6.89)$$

At this point, we once again require the equation to find the curvature of a graph, which can be manipulated and rewritten as follows:

$$k = \frac{x'y'' - y'x''}{(x'^2 + y'^2)^{3/2}}$$

$$k = \frac{\frac{d^2y}{dx^2}}{\left(1 + \left(\frac{dy}{dx}\right)^2\right)^{3/2}}$$

$$k = \frac{f''(x)}{\left(1 + (f'(x))^2\right)^{3/2}}$$

In substituting the values of the derivatives of $f(x)$ and $g(x)$, we can find the curvatures of the curves between the two fly-over waypoints. The equation of the curvature of $f(x)$ and $g(x)$ are shown below:

$$\text{curvature of graph } f(x) = \frac{f''(x)}{\left(1 + (f'(x))^2\right)^{3/2}}$$

curvature of graph $f(x)$

$$= \frac{(0.3601333059612x - 0.7994485774432)}{(1 + ((0.1800666529806x^2 - 0.7994485774432x + 0.9089627063146)^2)^{3/2}} \quad (1.35 < x < 4.1275)$$

$$\text{curvature of graph } g(x) = \frac{g''(x)}{(1 + (g'(x))^2)^{3/2}}$$

curvature of graph $g(x)$

$$= \frac{(0.9101990564574x - 5.0855750134908)}{(1 + ((0.4550995282287x^2 - 5.0855750134908x + 14.0830149746564)^2)^{3/2}} \quad (4.1275 < x < 6.89)$$

In order to visualise this however, we need to once again plot the curvatures on a graph. In this case, we let the values of our curvature κ be y , and the x values to be x . The results are shown below.

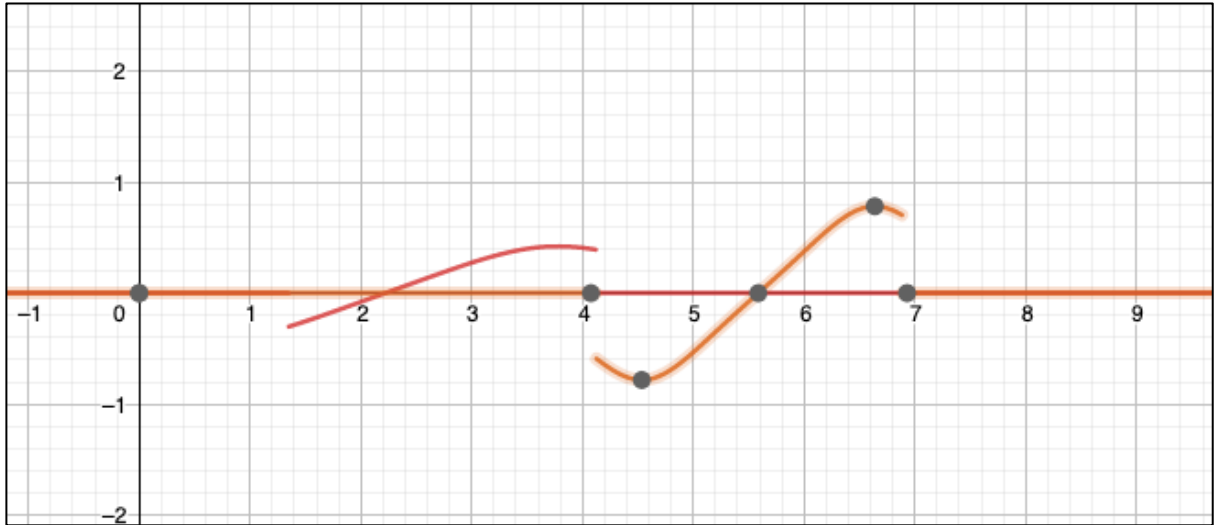


Figure 15 (Graph showing curvature against t for cubic curve, Author's own, 2022)

The graph demonstrates that the path has a much higher curvature than that of the Bezier curves, with a maximum curvature of ± 0.782656840712 when the graph is at its maximum. This indicates that the radius of curvature is also very small, at approximately 1.2 units, indicating a very sharp turn in the curve. Although it does pass through the x axis twice, meaning there were two instances of no curvature on the graph, the general curvature is still very high a lot of the time.

Conclusion:

This essay explores two methods of finding an ideal smooth path of construction for the purpose of finding the best route of an aircraft. The first method of investigation explores a complicated method involving Bezier curves, through linear interpolation. The second method however, explores a different method, which makes use of the GeoGebra “Fit” function to help create 2 cubic equations to plan an optimised route. From this research paper, in comparing the maximum values of the curvatures of the graphs, it is possible to tell that the Bezier curves allow for a much more optimised path compared to the cubic fit curves, and that Bezier curves have a consistently lower curvature. This means that if they are indeed used for autopilot systems, they would definitely allow for an increased in comfort for the passengers. This is because if a path does indeed have a much lower curvature, it also means that the aircraft will need to make a gentler turn, and will require lower bank angles (i.e. the angles in which a plane slants when making a turn). This in turn will allow for a more stable flight for passengers. Therefore, in conclusion, using Bezier curves appears to be better method for finding the most optimal path and can be used to plot the smoothest curve.

Extension:

One possible extension would be to combine piecewise linear functions and Bezier curves, for other purposes like route planning in autonomous driving vehicles. This is because as the number of points increase, and the Bezier functions increase, the deviation from the path itself may also increase. As such, by using an initial piecewise function, we will keep the path on the intended route. Bezier functions themselves will be used to smoothen out these linear piecewise functions, allowing for a much smoother curve.

References and Bibliography:

- “A Primer on Bézier Curves.” *Pomax.github.io*, 8 Jan. 2022,
<https://pomax.github.io/bezierinfo/#derivatives>
- “Calculator Suite.” *GeoGebra*, <https://www.geogebra.org/calculator>
- “Curvature (Article).” *Khan Academy*, Khan Academy,
<https://www.khanacademy.org/math/multivariable-calculus/multivariable-derivatives/differentiating-vector-valued-functions/a/curvature>
- “Curvature.” *From Wolfram MathWorld*, <https://mathworld.wolfram.com/Curvature.html>
- Google Maps*, Google, <https://maps.google.com/>
- khanacademy. “Curvature Formula, Part 1.” *YouTube*, YouTube, 20 May 2016,
<https://www.youtube.com/watch?v=gspjhwSNMWs>
- “Making Patterns: Pushing the Envelope.” *American Mathematical Society*, 2022,
<http://www.ams.org/publicoutreach/curve-stitching>
- “Order U.S. Department of Transportation 8260.44a CHG 1 Federal Aviation ...” *FEDERAL AVIATION ADMINISTRATION*, 2000,
<https://www.faa.gov/documentLibrary/media/Order/ND/8260.44A%20Chg1.pdf>
- Parametric Functions*, Xaktly, 2012,
[https://xaktly.com/ParametricEquations.html#:~:text=Parametric%20functions%3A%20Definition,t\)%2C%20and%20so%20on](https://xaktly.com/ParametricEquations.html#:~:text=Parametric%20functions%3A%20Definition,t)%2C%20and%20so%20on)
- Tank, Vivek. “How to Work with Bezier Curve in Games with Unity.” *Game Developer*, 6 Aug. 2018, <https://www.gamedeveloper.com/business/how-to-work-with-bezier-curve-in-games-with-unity>